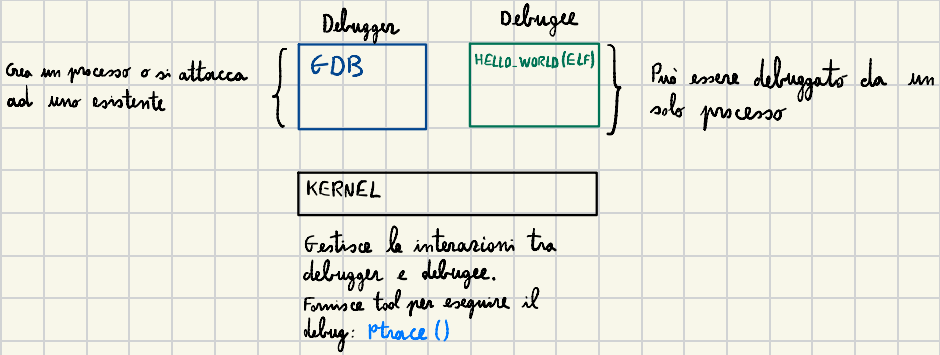
Debugging



Cos'è il Debug?

- Il debug è uno strumento che permette di ispezionare ogni processo, per:
  - risolvere problemi
  - exploitare le vulnerabilità
- I Security Engineers hanno sviluppato delle tecniche che permettono di rilevare quando un processo è sotto debug, in modo da cambiarne il comportamento. Questo è chiamato Anti-Debug

Debugging su Linux



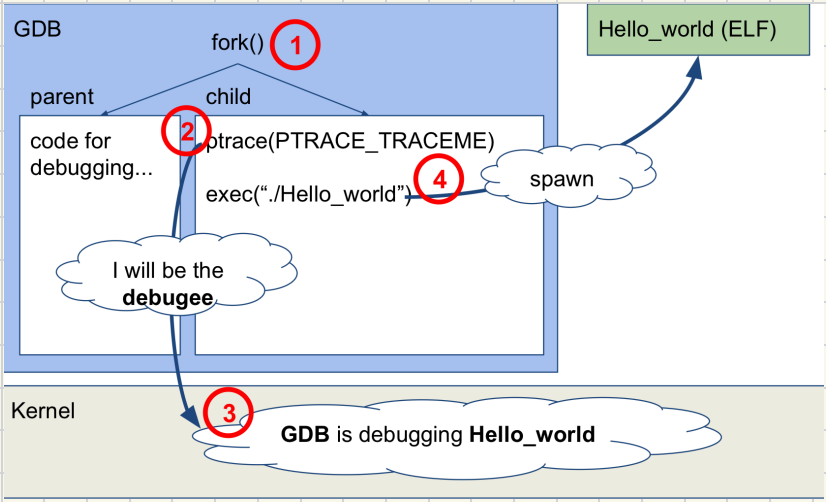
- Ptrace(): è una syscall di linux che permette ad un processo (Debugger) di ispezionare e controllare un altro processo (Debugee)
  es: step-to-step, cambio variabili, aggiunta breakpoint

Firma: long ptrace(enum __ptrace_request request, pid_t pid, void *addr, void *data);

ARGS:
- request: tipo di azione svolta dal debugger sul debugee (lettura/scrittura memoria, get/set valori registri)
- PID: processo su cui attaccarsi (Debugger può attaccarsi a se stesso)
- {addr, data}: usati per trasferire dati da/al Debugee

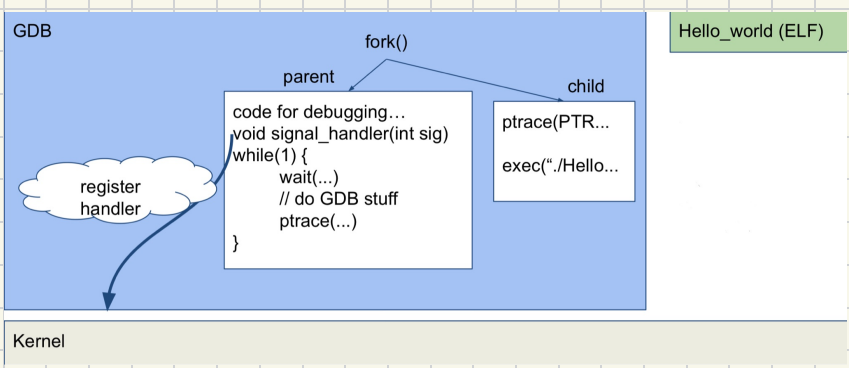
Returns: - TL; TR; -1 se qualcosa è andato storto, altrimenti dipende dalla richiesta

Spawning the Debugee Process



GDB avvia una fork: crea un sottoprocesso che eseguirà il Debugee, mentre GDB continua la sua esecuzione come debugger.

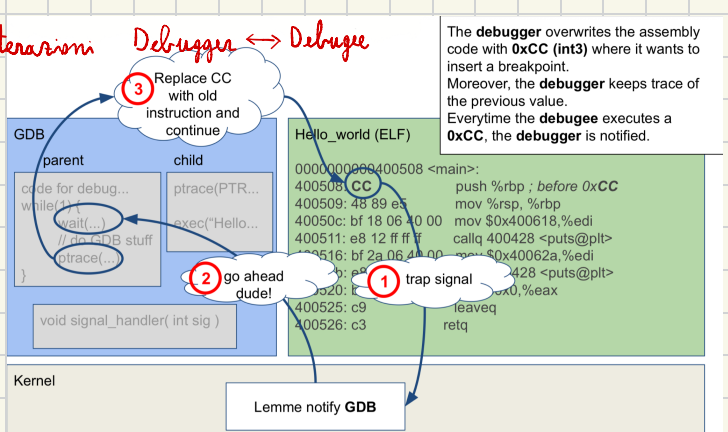
Registering Signal Handler



GDB resta in attesa di alcuni segnali dal Kernel.

Parent viene notificato dal Kernel quando il Debugee incontra un breakpoint

Interazioni Debugger ↔ Debugee



Recap Debugging

- Processo mediato dal Kernel
- Ptrace() è come un coltellino svizzero nel debugging
- Il debugger può debuggare più processi
- Il debugee può essere debuggato da un solo debugger
- Il debugger tiene traccia dei valori originali del debugee
- strace(), GDB, ecc. si basano su Ptrace() per funzionare

Anti-Debugging

1^ tecnica: check Ptrace() (Caso più comune)

```
if (Ptrace(PTRACE_TRACEME, 0, null, null) == -1) {
    Puts("there's already a debugger");
    return 1;
}
```

Se PTRACE_ME ritorna -1, qualcuno sta già debuggando il programma.

2^ tecnica: GDB env variable

```
if (getenv("LINES") || getenv("COLUMNS"))
    Puts("there's already a debugger");
```

ENV variables create da GDB, se esistono, GDB è in esecuzione

3^ tecnica: heap relocated

```
static unsigned char var_in_bss;
unsigned char *Probe = malloc(0x10);
if (Probe - var_in_bss < 0x20000) {
    Print("GDB in esecuzione");
}
```

GDB rialloca l'heap alla fine della sezione BSS. 0x20000 è la dimensione standard del BSS. Se dim(BSS) < 0x20000, l'heap è stato riallocato

4^ tecnica: GDB no-ASLR (Address Space Layout Randomization)

GDB alloca librerie e TEXT in indirizzi specifici (disabilita ASLR) che possono essere riconosciuti (es. base address di ELF e shared libraries).
Se questi elementi vengono trovati ad indirizzi specifici, GDB è attivo.

5^ tecnica: Who's my parent?

```
Pid_t Parent = getPPid();
link_target = read("/Proc/$Parent/exe");

if (!strcmp(basename(link_target), "gdb")
    res = RESULT_YES;
if (strstr(link_target, "lldb")
    res = RESULT_YES;
if (!strcmp(basename(link_target), "strace")
    res = RESULT_YES;
if (!strcmp(basename(link_target), "ltrace")
    res = RESULT_YES;
```

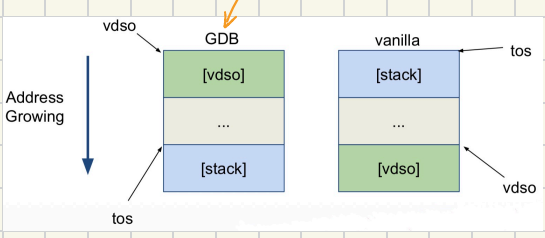
I processi parent possono essere ispezionati.
Se i parent sono: GDB, lldb, strace, ltrace o qualsiasi debugger, possiamo rilevarlo.

6^ tecnica: VDSO (Virtual Dynamic Shared Object)

```
unsigned long tos; // top of stack
unsigned long vdso = getauxval(AT_SYSINFO_EHDR);

if ((unsigned long)&tos > vdso)
    return RESULT_YES;
```

GDB sposta il VDSO sopra lo stack. Controllando gli indirizzi possiamo rilevare GDB



Come Bypassare l'Antidebug

- Patchare il file binario

- Main steps:
  1) cercare le tecniche di implementazione dell'anti-debug
     - check for ptrace(), 0xCC, getenv...
     - controllare anche fuori dal main thread (.init e .fini sections)

  2) Usare HEX Editor o Radare per patchare